

Testing de Software – Notas de Clase

Ubicación en el ciclo de vida

- El testing se ubica en la etapa de prueba del PDU.
- Antes se pensaba que solo podía hacerse con el producto ya programado. TDD lo anticipa.
- Consume entre el 30% y el 50% del presupuesto (50% en sistemas críticos).
- La implementación no debería superar el 33–35% del tiempo total del proyecto.

Requerimientos

- Deben ser objetivos, no ambiguos ni abiertos.
- Deben ser verificables (se puede demostrar si se cumplen o no).

Definición de Testing

- Actividad de control que contrasta el producto contra la documentación (ERS, requerimientos).
- Su objetivo es detectar defectos.
- Un testing exitoso es el que encuentra errores.
- No se debe decir: “Este producto no tiene defectos” ni “El testing asegura la calidad”.

Error vs Defecto vs Falla

- El testing detecta defectos, que fueron introducidos en etapas anteriores.
- Si el defecto se detecta en etapas posteriores, corregirlo es más caro.
- Falla: mal funcionamiento que afecta la satisfacción del cliente. No necesariamente implica que el sistema se caiga.

Automatización

- La automatización exige mayor especialización del tester.
- Se relaciona con:
 - CI (integración continua): pruebas de integración.
 - Delivery continuo: pruebas de sistema.
 - Deployment continuo: pruebas de aceptación.

Testing metodológico vs Ad Hoc

- Ad Hoc: pruebas sin planificación. Difícil reproducibilidad. No sirve.
- Metodológico: sigue un proceso formal.

Proceso de Testing

1. Planificación y diseño
 - Plan de pruebas
 - Casos de prueba: paso a paso detallado, con datos específicos, escenario y resultado esperado.
 - Se pueden diseñar antes de la implementación, usando los requerimientos como entrada (similar a TDD).
 2. Ejecución
 - Se ejecutan los casos.
 - Si hay errores, se genera un reporte de defectos.
 - El tester no corrige, solo detecta.
 3. Seguimiento y cierre
 - Se hace seguimiento del reporte y se cierra el ciclo.
- El 50% del esfuerzo está en la planificación y diseño, el otro 50% en ejecución y cierre.
 - Ciclo de prueba: cada ejecución del proceso. El primero se llama ciclo 0.
 - Si hay cambios en la planificación, se vuelve a esa etapa.

Estrategias de regresión

- Sin regresión: se prueban solo los casos que fallaron.
- Con regresión: se prueban todos los casos, por si se introdujeron nuevos errores.

Verificación vs Validación

- Verificación: que funcione correctamente.
- Validación: que cumpla con lo que necesita el cliente.
- Sin requerimientos definidos, no se puede validar.

Niveles de prueba

1. Prueba unitaria: la hace el desarrollador.
2. Prueba de integración: la hace testing.
3. Prueba de sistemas o versión: la hace testing.
4. Prueba de aceptación de usuario: la hace el usuario (PO en ágil).

Ambientes de prueba

- Desarrollo: pruebas unitarias (a veces integración).
- Prueba: integración y sistemas. Los desarrolladores no deben entrar.
- Preproducción: ideal para pruebas de aceptación. No siempre se implementa por costos.
- Producción: transacciones reales por parte del usuario.

Tipos de pruebas

Sobre requerimientos funcionales

- Requerimientos de negocio
- Funcionalidad (operación normal y pruebas negativas)
- Documentación (manuales de usuario)

Sobre requerimientos no funcionales

- Stress: probar por encima de lo prometido.
- Escala completa: probar lo prometido.
- Interfaces: se mide por porcentaje de cobertura.
- Seguridad: buscar vulnerabilidades.
- Performance: tiempo de respuesta.
- Volumen de datos: probar con distintas cargas.
- Smoke test: detectar errores invalidantes.
- Exploratorias: entender el sistema cuando no hay documentación.

Severidad (definida por el tester)

- Bloqueante/invalidante: el sistema se clava.
- Grave: mal funcionamiento (ej. sumar mal).
- Leve: errores en formatos o capacidad.
- Cosmética: errores visuales o ortográficos.

Prioridad (definida por el cliente)

- Alta/urgente
- Media
- Baja